

Composite template

```
// #pragma comment(linker, "/stack:20000000")
// #pragma GCC optimize("Ofast,O3,unroll-loops")
// #pragma GCC target("sse,sse2,sse3,ssse3,sse4,abm,mmx,avx2,bmi,bmi2,popcnt,lzcnt")
// assert(__builtin_cpu_supports("sse2"));

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
template<class T, class comp=less<T>> using ordered_set = __gnu_pbds::tree<T,
__gnu_pbds::null_type, comp, __gnu_pbds::rb_tree_tag,
__gnu_pbds::tree_order_statistics_node_update>;
// find_by_order(k) - iterator on k's elements
// order_of_key(x) - number of elements, strictly less than x

#define dbg(...) {auto t = std::make_tuple(__VA_ARGS__); cerr << "\033[33m" <<
(#_VA_ARGS_) << ": "; dbg_vals(t, make_index_sequence<tuple_size_v<decltype(t)>>{}); cerr
<< "\033[0m\n";}

template <typename T, typename=decltype(declval<T>().begin()),
typename=enable_if_t<!is_convertible_v<T, string_view>>> ostream& operator<<(ostream& out,
const T& x){ for (const auto& i : x) out << i << ' '; return out;}

template<class Tuple, size_t... I> void dbg_vals(const Tuple& t, index_sequence<I...>){ auto
sep=""; ((cerr<<sep<<"\033[31m"<<get<I>(t)<<"\033[0m", sep=" | "),...); }

mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
inline int rng(int l, int r){return (rnd() % (r - l + 1) + l);}
ll rll(ll r=numeric_limits<ll>::max(), ll l=0){ /*random long long*/ return
uniform_int_distribution<ll>(l, r)(rng); }
template <class T> bool emin(T& a, const T& b) { return b < a ? a = b, 1 : 0; }
template <class T> bool emax(T& a, const T& b) { return a < b ? a = b, 1 : 0; }
template <class T, class U> void erase(T& t, const U& u) {
    auto it = t.find(u);
    if (it != t.end()) t.erase(it);
}
inline namespace FileIO {
    void setIn(string s) { freopen(s.c_str(),"r",stdin); }
    void setOut(string s) { freopen(s.c_str(),"w",stdout); }
    void setIO(string s = "") {
        ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
        if (s.ss) setIn(s+".in"), setOut(s+".out");
    }
}
```

VS Code template

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "type": "process",
            "label": "Compile&Run",
            "command": [ "cmd.exe", ],
            "args": [
                "/c g++.exe ${file} -o ${fileBasenameNoExtension}.exe -std=c++17 -O2 &&
                ${fileBasenameNoExtension}.exe < in",
            ],
        }
    ]
}
```

```

        "problemMatcher": ["$gcc"],
        "group": {
            "kind": "build",
            "isDefault": true
        },
        "presentation": {
            "focus": true,
            "showReuseMessage": false,
            "clear": true
        }
    }
}

]
}
"/c g++.exe ${file} -o ${fileBasenameNoExtension}.exe -std=c++17 -Wall -Wextra -pedantic
-Wformat=2 -Wfloat-equal -Wlogical-op -Wshift-overflow=2 -Wduplicated-cond -Wcast-qual
-Wcast-align -D_GLIBCXX_DEBUG -D_GLIBCXX_DEBUG_PEDANTIC -D_FORTIFY_SOURCE=2
-fstack-protector -Wl,--stack=134217728 -DLOCAL && ${fileBasenameNoExtension}.exe < in",

```

Extended GCD

```

array<ll, 3> extgcd(ll a, ll b) {
    if (!b) return {a, 1, 0};
    auto [d, y, x] = extgcd(b, a % b);
    return {d, x, y - a/b * x};
} // {gcd, x, y} such that a*x + b*y = gcd(a, b)

```

SOS DP

```

for(int i = 0; i < (1<<N); ++i)
    F[i] = A[i];
for(int i = 0; i < N; ++i) for(int mask = 0; mask < (1<<N); ++mask) {
    if(mask & (1<<i))
        F[mask] += F[mask^(1<<i)];
}

```

Matching

```

vi Blossom(vector<vi>& graph) {
    int n = graph.size(), T = -1;
    vi mate(n, -1), label(n), par(n), orig(n), aux(n, -1), q;
    auto lca = [&](int x, int y) {
        for (T++; ; swap(x, y)) {
            if (x == -1) continue;
            if (aux[x] == T) return x;
            aux[x] = T;
            x = (mate[x] == -1 ? -1 : orig[par[mate[x]]]);
        }
    };
    auto blossom = [&](int v, int w, int a) {
        while (orig[v] != a) {
            par[v] = w; w = mate[v];
            if (label[w] == 1) label[w] = 0, q.push_back(w);
            orig[v] = orig[w] = a, v = par[w];
        }
    };
    auto augment = [&](int v) {
        while (v != -1) {
            int pv = par[v], nv = mate[pv];
            mate[v] = pv, mate[pv] = v, v = nv;
        }
    };
}

```

```

    }
};
auto bfs = [&](int root) {
    fill(all(label), -1), iota(all(orig), 0);
    q.clear(), q.push_back(root), label[root] = 0;
    for (int i = 0; i < sz(q); i++) {
        int v = q[i];
        for (auto x : graph[v])
            if (label[x] == -1) {
                label[x] = 1, par[x] = v;
                if (mate[x] == -1) return augment(x);
                label[mate[x]] = 0, q.push_back(mate[x]);
            } else if (label[x] == 0 && orig[v] != orig[x]) {
                int a = lca(orig[v], orig[x]);
                blossom(x, v, a), blossom(v, x, a);
            }
    }
};
// Time halves if you start with (any) maximal matching.
for (int i = 0; i < n; i++)
    if (mate[i] == -1)
        bfs(i);
return mate;
}

vector<pair<int,int>> matching_bipartite(vll gr[], int n){
    vector<char> is_l(n, -1);
    vector<int> q(n), dist(n), match(n, -1), ptr(n);
    for(int i = 0; i < n; i++){
        if (is_l[i] != -1) continue;
        int l = 0, r = 0;
        q[r++] = i, is_l[i] = 1;
        while(l < r){
            int x = q[l++];
            for(auto v : gr[x]) if (is_l[v] == -1) is_l[v] = !is_l[x], q[r++] = v;
        }
    }
    for(int i = 0; i < n; i++) if (is_l[i]) match[i] = 0;
    function<bool(int)> kuhn = [&](int x) -> bool {
        for(int &i = ptr[x]; i < int(gr[x].size()); i++){
            if (is_l[gr[x][i]]) continue;
            int &u = match[gr[x][i]];
            if (u == -1 || (dist[u] == dist[x] + 1 && kuhn(u))) return (match[u = x] =
true);
        }
        return false;
    };
    while(1){
        fill(dist.begin(), dist.end(), n);
        int l = 0, r = 0, flag = false, x, u;
        for(int i = 0; i < n; i++) if (is_l[i] && !match[i]) dist[i] = 0, q[r++] = i;
        while(l < r){
            if (is_l[x = q[l++]]) for(auto v : gr[x]){
                if (is_l[v]) continue;
                flag |= ((u = match[v]) == -1);
                if (u != -1 && dist[x] + 1 < dist[u]) dist[u] = dist[x]+1, q[r++] = u;
            }
        }
    }
}

```

```

    }
    if (!flag) break;
    fill(ptr.begin(), ptr.end(), 0);
    for(int i = 0; i < n; i++) if (is_l[i] && !match[i]) kuhn(i);
}
vector<pair<int, int>> ans;
for(int i = 0; i < n; i++) if (!is_l[i] && match[i] != -1) ans.emplace_back(match[i],
i);
return ans;
}

```

Fenwick tree

```

update(ll i, ll num) for(; i < n; i |= i+1) fen[i] = (fen[i] + num);
ll get(ll r) for(; r >= 0; r = (r & (r+1))-1) ans = (ans + fen[r]);

```

Strings

```

class Manacher{
    vector<int> d1;
public:
    template<class T=string, class TT=char>
    Manacher(const T& x, TT d){
        T s(x.size()*2, d);
        for(size_t i = 0; i < x.size(); i++) s[2*i] = x[i];
        d1.assign(s.size(), 0);
        int l = 0, r = -1;
        for(int i = 0; i < (int)s.size(); i++){
            int& k = d1[i];
            k = (i > r ? 1 : min(d1[l+r-i], r-i+1));
            while(i+k < sz(s) && i-k >= 0 && s[i+k] == s[i-k]) k++;
            if (i+k-1 > r) l = i-k+1, r = i+k-1;
        }
    }
    int get(int pos, bool odl=true){ return (odl ? d1[pos*2]+1 : d1[pos*2+1])/2; }
    bool is_pal(int l, int r){ int len = r-l+1; return get((l+r)>>1, len&1)*2 >= len; }
};

vector<int> prefix_function (string s) {
    vector<int> pi (s.size());
    for (int i=1; i<s.size(); ++i) {
        int j = pi[i-1];
        while (j > 0 && s[i] != s[j]) j = pi[j-1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}

vector<int> z_function (string s) {
    vector<int> z (s.size());
    for (int i=1, l=0, r=0; i<s.size(); ++i) {
        if (i <= r) z[i] = min (r-i+1, z[i-l]);
        while (i+z[i] < n && s[z[i]] == s[i+z[i]]) ++z[i];
        if (i+z[i]-1 > r) l = i, r = i+z[i]-1;
    }
    return z;
}

array<vector<int>, 2> SuffixArray(string& s) { // or basic_string<int>
    int n = sz(s) + 1, k = 0, a, b, lim = *max_element(all(s))+1;

```

```

vector<int> x(all(s)), y(n), ws(max(n, lim)), sa(n), lcp(n);
x.push_back(0), iota(all(sa), 0);
for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
    p = j, iota(all(y), n - j);
    fi(n) if (sa[i] >= j) y[p++] = sa[i] - j;
    fill(all(ws), 0);
    fi(n) ws[x[i]]++;
    fio(lim) ws[i] += ws[i - 1];
    for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
    swap(x, y), p = 1, x[sa[0]] = 0;
    fio(n) a = sa[i - 1], b = sa[i], x[b] = (y[a] == y[b] && y[a + j] == y[b + j]) ? p -
1 : p++;
}
for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
    for (k && k--, j = sa[x[i] - 1]; s[i + k] == s[j + k]; k++);
sa.erase(sa.begin()); lcp.erase(lcp.begin());
return {sa, lcp};
}

class AhoCorasick{
    struct Node{
        int sufflink = -1;
        map<char, int> next;
    };
public:
    vector<Node> t;
    AhoCorasick(ll n){ t.emplace_back(); t.reserve(n); }
    int add(string s, int root=0){
        for(char c : s){
            if (!t[root].next.count(c)){
                t[root].next[c] = t.size();
                t.emplace_back();
            }
            root = t[root].next[c];
        }
        return root;
    }
    void build(){
        queue<int> q;
        q.push(0);
        while(q.size()){
            int v = q.front(); q.pop();
            for(auto [c, nex] : t[v].next){
                int link = t[v].sufflink;
                while(link != -1 && !t[link].next.count(c)) link = t[link].sufflink;
                t[nex].sufflink = (link == -1 ? 0 : t[link].next[c]);
                q.push(nex);
            }
        }
    }
    int next(int v, char c){
        while(!t[v].next[c]){
            if (!v) return v;
            v = t[v].sufflink;
        }
        return t[v].next[c];
    }
}

```

};

CHT

```
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

template<bool is_max=true>
struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { /*floored division*/ return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        if (!is_max) k = -k, m = -m;
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll query(ll x) { if (empty()) return INF;
        auto l = *lower_bound(x);
        return (l.k * x + l.m) * (is_max ? 1 : -1);
    }
};
```

Li Chao Tree

```
void add_line(Line nw, Line *v, ll l, ll r){
    ll m = (l + r) / 2;
    bool lef = nw(l) < v(l);
    bool mid = nw(m) < v(m);
    if (mid) swap_lines(nw, *v);
    if (r - l <= 1) return;
    if (lef != mid) add_line(nw, v->to_left(), l, m);
    else
        add_line(nw, v->to_righ(), m, r);
}
```

Bridges and articulation points

```
void dfs (int v, int p = -1) {
    used[v] = true, tin[v] = fup[v] = timer++;
    for (int to : g[v]) {
        if (to == p) continue;
        if (used[to]) fup[v] = min (fup[v], tin[to]);
        else {
            dfs (to, v);
            fup[v] = min (fup[v], fup[to]);
            if (fup[to] > tin[v]) IS_BRIDGE(v,to);
            if (fup[to] >= tin[v] && p != -1) IS_CUTPOINT(v);
            ++children;
        }
    }
}
```

```

    }
}
if (p == -1 && children > 1) IS_CUTPOINT(v);
}

```

HLD

```

struct HLD{
    int s[MAXN], p[MAXN], tin[MAXN], tout[MAXN], head[MAXN], levs[MAXN], t;
    bool upper(int a, int b){ return tin[a] <= tin[b] && tin[b] < tout[a]; }
    HLD(vector<ll> gr[]){
        function<void(int, int, int)> sizes = [&](int x, int par, int lev){
            s[x] = 1, p[x] = par, levs[x] = lev;
            if (gr[x].size() && gr[x][0] == par) swap(gr[x][0], gr[x].back());
            for(ll& i : gr[x]){
                if (i == par) continue;
                sizes(i, x, lev+1);
                s[x] += s[i];
                if (s[i] > s[gr[x][0]]) swap(i, gr[x][0]);
            }
        }; sizes(0, 0, 0);
        function<void(int)> hld = [&](int x){
            tin[x] = t++;
            for(ll i : gr[x]){
                if (i == p[x]) cn;
                head[i] = (i == gr[x][0] ? head[x] : i);
                hld(i);
            }
            tout[x] = t;
        }; t = 0; hld(0);
    }
    int do_path(int a, int b, function<void(int, int)> f){
        for(; head[a] != head[b]; b = p[head[b]]){
            if (levs[head[a]] > levs[head[b]]) swap(a, b);
            f(tin[head[b]], tin[b]);
        }
        if (tin[a] > tin[b]) swap(a, b);
        f(tin[a], tin[b]);
        return a;
    }
    void do_subtree(int v, function<void(int, int)> f){ f(tin[v], tout[v]-1); }
    int lca(ll a, ll b){ return do_path(a, b, [](int a, int b){}); }
    ll dist(ll a, ll b){ return levs[a] + levs[b] - 2*levs[lca(a, b)]; }
};

```

Centroid

```

void build(int v, int lvl, int pr = -1){
    dfs(v, -1);
    int cen = v;
    while(2 * rz[mx[cen]] > rz[v]) cen = mx[cen];
    lev[cen] = lvl; p[cen] = pr;
    us[cen] = 1;
    for(auto to: g[cen]){
        if(us[to]) continue;
        build(to, lvl + 1, v);
    }
}

```

Treap

```

void update(int v){
    push(t[v].l); push(t[v].r);
    t[v].sz = t[t[v].l].sz + t[t[v].r].sz + 1;
    int x, y;
    if(t[v].l == 0) x = INF;
    else x = t[t[v].l].mn;
    if(t[v].r == 0) y = INF;
    else y = t[t[v].r].mn;
    t[v].mn = min(min(x, y), t[v].val);
}

pii split(int n, int k){
    if(n == 0) return {0, 0};
    push(n);
    if(t[t[n].l].sz + 1 <= k){
        pii res = split(t[n].r, k - t[t[n].l].sz - 1);
        t[n].r = res.F;
        update(n);
        return {n, res.S};
    }
    else{
        pii res = split(t[n].l, k);
        t[n].l = res.S;
        update(n);
        return {res.F, n};
    }
}

int merg(int t1, int t2){
    push(t1); push(t2);
    if(t1 == 0) return t2;
    if(t2 == 0) return t1;
    if(t[t1].y > t[t2].y){
        int res = merg(t[t1].r, t2);
        t[t1].r = res;
        update(t1);
        return t1;
    }
    else{
        int res = merg(t1, t[t2].l);
        t[t2].l = res;
        update(t2);
        return t2;
    }
}

```

Dinic

```

struct Dinic {
    struct FlowEdge {
        int v, u;
        ll cap, flow = 0;
        FlowEdge(int v, int u, ll cap) : v(v), u(u), cap(cap) {}
        FlowEdge() {}
    };

```

```

FlowEdge edges[MAXN*3];
vector<int> adj[MAXN];
int n, m = 0, s, t;
int level[MAXN], ptr[MAXN];
queue<int> q;
void init(int _n, int _s, int _t) {
    n = _n, s = _s, t = _t, m = 0;
    for (int i = 0; i < n; i++) adj[i].clear();
}
void add_edge(int v, int u, int cap) {
    edges[m] = FlowEdge(v, u, cap);
    edges[m+1] = FlowEdge(u, v, 0);
    adj[v].push_back(m);
    adj[u].push_back(m + 1);
    m += 2;
}
bool bfs() {
    while (!q.empty()) {
        int v = q.front();
        if (v == t) break;
        q.pop();
        for (int id : adj[v]) {
            if (edges[id].cap == edges[id].flow) continue;
            if (level[edges[id].u] >= level[s]) continue;
            level[edges[id].u] = level[v] + 1;
            q.push(edges[id].u);
        }
    }
    return level[t] >= level[s];
}
int dfs(int v, int pushed) {
    if (pushed == 0) return 0;
    if (v == t) return pushed;
    for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
        int id = adj[v][cid];
        int u = edges[id].u;
        if (level[v] + 1 != level[u]) continue;
        int tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
        if (tr == 0) continue;
        edges[id].flow += tr;
        edges[id ^ 1].flow -= tr;
        return tr;
    }
    return 0;
}
int flow() {
    int f = 0;
    while (1) {
        level[s] += n+5;
        q.push(s);
        if (!bfs()) break;
        fill(ptr, ptr+n, 0);
        while (int pushed = dfs(s, INF)) f += pushed;
    }
    return f;
}
} flow;

```

KUHN (general)

```
bool try_kuhn(int v) {
    if(us[v]) return 0;
    us[v] = 1;
    for(auto to: g[v]){
        if(mt[to] == -1 || try_kuhn(mt[to])){
            mt[to] = v;
            return 1;
        }
    }
    return 0;
}
```

Eulerian Trail

```
template<bool directed=false>
class Eulerian_Path{
    vector<vector<array<int, 2>>> gr;
    int m;
public:
    Eulerian_Path(int n): gr(n), m(0) {}
    void add_edge(int a, int b){
        gr[a].push_back({b, m++});
        if constexpr (!directed) gr[b].push_back({a, m-1});
    }
    optional<array<vector<int>, 2>> get_res(){
        int start = -1, end = -1;
        if constexpr (directed){
            vector<int> indeg(gr.size(), 0);
            for(auto& ed : gr) for (auto [v, _] : ed) indeg[v]++;
            for(size_t i = 0; i < gr.size(); i++){
                if ((int)gr[i].size()+1 == indeg[i]){
                    if (start != -1) return {};
                    start = i;
                }else if ((int)gr[i].size() == indeg[i]+1){
                    if (end != -1) return {};
                    end = i;
                }else if ((int)gr[i].size() != indeg[i]) return {};
            }
        } else{
            for(size_t i = 0; i < gr.size(); i++){
                if (gr[i].size() % 2){
                    if (start == -1) start = i;
                    else if (end == -1) end = i;
                    else return {};
                }
            }
        }
        if (start != -1){
            if (end == -1) return {};
            add_edge(start, end);
        } else {
            start = 0;
            for(size_t i = 0; i < gr.size(); i++) if (gr[i].size()) start = i;
        }
        vector<char> active_edge;
```

```

    if constexpr (!directed) active_edge.assign(m, 1);
    vector<array<int, 2>> st{{start, -1}};
    vector<int> rese, resv, it(gr.size(), 0);
    st.reserve(m+1), rese.reserve(m), resv.reserve(m+1);
    while(st.size()){
        int v = st.back()[0];
        if constexpr (!directed)
            while(it[v] < (int)gr[v].size() && !active_edge[gr[v][it[v]][1]]) it[v]++;
        if (it[v] < (int)gr[v].size()){
            if constexpr (!directed) active_edge[gr[v][it[v]][1]] = 0;
            st.push_back(gr[v][it[v]++]);
        }else {
            if (st.back()[1] != -1) rese.push_back(st.back()[1]);
            st.pop_back(), resv.push_back(v);
        }
    }
    if (m != (int)rese.size()) return {};
    if constexpr (directed) reverse(resv.begin(), resv.end()), reverse(rese.begin(),
rese.end());
    if (end != -1){ // path
        resv.pop_back();
        for(size_t i = 1; i < rese.size(); i++){
            if (rese[i-1] == m-1){
                rotate(resv.begin(), resv.begin()+i, resv.end());
                rotate(rese.begin(), rese.begin()+i, rese.end());
                rese.pop_back();
                break;
            }
        }
    }
    return {{resv, rese}};
}
};

```

Геометрия (а конкретнее сравнение векторов по углам)

```

struct point
{
    int x, y;
    ld distp(point p2){
        return hypotl(p2.x - x, p2.y - y);
    }
} p[N];

struct vct{
    point p1, p2;
    int x, y, i, j;
    ld dist;
    void make(){
        p1 = p[i];
        p2 = p[j];
        x = p2.x - p1.x;
        y = p2.y - p1.y;
        dist = p1.distp(p2);
    }
    bool hp(){
        return y < 0 || (y == 0 && x < 0);
    }
}

```

```

    }
    int vect_prod(vct v2){
        return x * v2.y - y * v2.x;
    }
    bool operator<(vct v2){
        if (this -> hp() != v2.hp()) return this -> hp() < v2.hp();
        int prod = x * v2.y - y * v2.x;
        if (prod == 0){
            return (dist < v2.dist);
        }
        return prod > 0;
    }
};

```

2SAT

```

class Two_SAT{
    vector<vector<int>>> gr;
    int code(ll v){ return v > 0 ? 2*v-2 : -2*v-1; } // negative v = !v
public:
    Two_SAT(ll n) { gr.resize(2*n); }
    bool has_ans(vll& vals){
        int timer = 0, n = gr.size()/2;
        vals.assign(n, -1);
        vector<int> tout(2*n, 0), st, comps(2*n, 0);
        function<int(int)> dfs = [&](int x) -> int {
            int low = tout[x] = ++timer, v; st.push_back(x);
            for(int v : gr[x]) if (!comps[v]) low = min(low, tout[v] ? tout[v] : dfs(v));
            if (low == tout[x])do{
                v = st.back(); st.pop_back();
                comps[v] = low;
                if (vals[v/2] == -1) vals[v/2] = !(v&1);
            } while(v != x);
            return tout[x] = low;
        };
        for(int i = 0; i < 2*n; i++) if (!comps[i]) dfs(i);
        for(int i = 0; i < n; i++) if (comps[2*i] == comps[2*i+1]) return false;
        return true;
    }
    void add_impl(ll a, ll b) /* a -> b */ { gr[code(a)].pb(code(b)); }
    void add_or(ll a, ll b) /* a | b */ { add_impl(-a, b), add_impl(-b, a); }
    void add_xor(ll a, ll b) /* a ^ b */ { add_or(a, b), add_or(-a, -b); }
    void set_val(ll v) /* vals[abs(v)] = (v > 0) */ { add_or(v, v); }
};

```

Sparse Table

```

template <class Node>
class Sparse{
    const static ll K = 20;
    Node vals[K][MAXN];
    int log2_floor(unsigned long long i) {
        return i ? __builtin_clzll(1) - __builtin_clzll(i) : -1;
    }
public:
    Sparse(ll ar[], ll n){
        for(ll i = 0; i < n; i++) vals[0][i] = Node(ar[i], i);
        for(ll i = 1; i < K; i++){

```

```

        for(ll j = 0; j + (1ll<<i) <= n; j++){
            vals[i][j] = vals[i-1][j] + vals[i-1][j + (1ll<<(i-1))];
        }
    }
}
Node get(ll l, ll r){
    ll i = log2_floor(r-l+1);
    return vals[i][l] + vals[i][r-(1ll<<i)+1];
}
};

```

Primality check

```

template<typename big_uint=ull>
bool MillerRabin_uint(ull n) { // returns true if n is prime, else returns false.
    if (n < 2) return false;
    for(ull a : {2, 3, 5, 13, 19, 73, 193, 407521, 299210837}) if (n == a) return true;
    ull s = __builtin_ctzll(n-1), d = (n-1)>>s;
    for (ull a : {2, 3, 5, 7}) { // {2, 325, 9375, 28178, 450775, 9780504, 1795265022}
        if (n == a) return true; // not
        ull x = 1, p = d;
        for(ull p=d; p; a=(big_uint)a*a%n, p>>=1) if (p&1) x = (big_uint)x*a%n;
        if (x == 1 || x == n-1) continue;
        if (s <= 1) return false;
        for(ull r = 1; r < s; r++){
            if ((x = (big_uint)x*x%n) == n-1) break;
            if (r+1 == s) return false;
        }
    }
    return true;
}
// Another impl
ull modmul(ull a, ull b, ull M) { return (LL(a) * b) % M; }
ull modpow(ull b, ull e, ull mod) { ull ans = 1;
    for (; e; b = modmul(b, b, mod), e >>= 1) if (e & 1) ans = modmul(ans, b, mod);
    return ans; }
// 1e5 tests, n <= 1e18, time = 341 ms
bool is_prime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
    ull s = __builtin_ctzll(n - 1), d = n >> s;
    for (ull a : A) {
        ull p = modpow(a % n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--) p = modmul(p, p, n);
        if (p != n - 1 && i != s) return false; } return true; }
// sometimes incorrect
// 100 tests, n <= 1e18, time = 280 ms (sometimes > 10 s)
ull pollard(ull n) {
    auto f = [n](ull x) { return modmul(x, x, n) + 1; };
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) { x = ++i; y = f(x); }
        if ((q = modmul(prd, max(x, y) - min(x, y), n))) prd = q;
        x = f(x); y = f(f(y));
    }
    return __gcd(prd, n);
}
// 100 tests, n <= 1e18, time = 1432 ms

```

```

ull pollard(ull n) { // correct but slower
    if (is_prime(n)) return n; if (n % 2 == 0) return 2; ull s = 0;
    auto f = [&](ull x) { return (LL(x) * x + s) % n; };
    while (true) {
        s++; ull x = s, y = f(x);
        while (true) {
            ull d = gcd(y - x + n, n); if (d == 0 || d == n) break;
            if (d != 1) return d;
            x = f(x); y = f(f(y));
        }
    }
}

```

Non-recursive simple segment tree

```

template <typename T>
struct segment_tree {
    static const int N = 1e6 + 10;
    const T inf = numeric_limits<T>::max() / 2;
    T t[2 * N + 10];
    void build() {
        for (int i = N - 1; i > 0; i--) {
            t[i] = t[i << 1] + t[i << 1 | 1];
        }
    }
    void update(int pos, T val) {
        for (t[pos += N] = val; pos > 1; pos >>= 1) {
            t[pos >> 1] = t[pos] + t[pos ^ 1];
        }
    }
    T get(int l, int r) { // !!!! [L, R) !!!!
        T res = 0;
        for (l += N, r += N; l <= r; l >>= 1, r >>= 1) {
            if (l & 1) res += t[l++];
            if (!(r & 1)) res += t[r--];
        }
        return res;
    }
};

```

Convex hull of points

```

template<typename T>
vector<Point<T>> convex_hull(vector<Point<T>> v) {
    int n = v.size(), k = 0;
    if (n < 3) return v;
    vector<Point<T>> hull(2 * n);
    sort(v.begin(), v.end());
    for (int i = 0; i < n; i++) {
        while (k >= 2 && cross(hull[k - 2], hull[k - 1], v[i]) <= 0) k--;
        hull[k++] = v[i];
    }
    for (int i = n - 1, t = k + 1; i > 0; i--) {
        while (k >= t && cross(hull[k - 2], hull[k - 1], v[i - 1]) <= 0) k--;
        hull[k++] = v[i - 1];
    }
    hull.resize(k - 1);
    return hull;
}

```

}

Floor sum

```
// @param n `n < 2^32`
// @param m `1 <= m < 2^32`
// @return sum_{i=0}^{n-1} floor((ai + b) / m) (mod 2^64)
unsigned long long floor_sum_unsigned(ull n, ull m, ull a, ull b) {
    ull ans = 0;
    while (true) {
        if (a >= m) {
            ans += n * (n - 1) / 2 * (a / m);
            a %= m;
        }
        if (b >= m) {
            ans += n * (b / m);
            b %= m;
        }

        unsigned long long y_max = a * n + b;
        if (y_max < m) break;
        // y_max < m * (n + 1)
        // floor(y_max / m) <= n
        n = (unsigned long long)(y_max / m);
        b = (unsigned long long)(y_max % m);
        std::swap(m, a);
    }
    return ans;
}
```

DP Optimizations

name	recurrence	conditions	time
CHT-1	$\text{emin}(\text{dp}[i], \text{dp}[j] + b[j] * a[i])$	$b[j] \geq b[j+1]$	$O(n \log n)$
CHT-2	$\text{emin}(\text{dp}[i][j], \text{dp}[i-1][k] + b[k] * a[j])$	$b[k] \geq b[k+1], a[j] \leq a[j+1]$	$O(kn)$
D&C	$\text{emin}(\text{dp}[i][j], \text{dp}[i-1][k] + C(k, j))$	$A[i][j] \leq A[i][j+1]$	$O(kn \log n)$
Knuth	$\text{emin}(\text{dp}[i][j], \text{dp}[i][k] + \text{dp}[k][j] + C(i, j))$	$A[i][j-1] \leq A[i][j] \leq A[i+1][j]$	$O(n^2)$
Aliens	$\text{emin}(\text{dp}[i][k], \text{dp}[j][k-1] + C(j+1, i) + \lambda)$	$\text{cnt}(\lambda)$ is monotone	$O(n \log n \log C)$

For Knuth optimization also:

- * quadrangle inequality: $C(a, c) + C(b, d) \leq C(a, d) + C(b, c), a \leq b \leq c \leq d$
- * monotonicity: $C(b, c) \leq C(a, d), a \leq b \leq c \leq d$

$A[i][j]$ - the smallest k that gives optimal answer, $C(i, j)$ - cost function, λ - penalty, $\text{cnt}(\lambda)$ - number of partitions

Divide and Conquer DP

```
int m, n;
vector<long long> dp_before, dp_cur;
long long C(int i, int j);
// compute dp_cur[l], ... dp_cur[r] (inclusive)
void compute(int l, int r, int optl, int optl) {
    if (l > r)
        return;
    int mid = (l + r) >> 1;
    pair<long long, int> best = {LLONG_MAX, -1};
    for (int k = optl; k <= min(mid, optl); k++) {
        best = min(best, {(k ? dp_before[k - 1] : 0) + C(k, mid), k});
    }
}
```

```

    }
    dp_cur[mid] = best.first;
    int opt = best.second;
    compute(l, mid - 1, optl, opt);
    compute(mid + 1, r, opt, optr);
}
long long solve() {
    dp_before.assign(n,0);
    dp_cur.assign(n,0);
    for (int i = 0; i < n; i++)
        dp_before[i] = C(0, i);
    for (int i = 1; i < m; i++) {
        compute(0, n - 1, 0, n - 1);
        dp_before = dp_cur;
    }
    return dp_before[n - 1];
}

```

Knuth's Optimization

```

int solve() {
    int N;
    ... // read N and input
    int dp[N][N], opt[N][N];
    auto C = [&](int i, int j) {
        ... // Implement cost function C.
    };
    for (int i = 0; i < N; i++) {
        opt[i][i] = i;
        ... // Initialize dp[i][i] according to the problem
    }
    for (int i = N-2; i >= 0; i--) {
        for (int j = i+1; j < N; j++) {
            int mn = INT_MAX;
            int cost = C(i, j);
            for (int k = opt[i][j-1]; k <= min(j-1, opt[i+1][j]); k++) {
                if (mn >= dp[i][k] + dp[k+1][j] + cost) {
                    opt[i][j] = k;
                    mn = dp[i][k] + dp[k+1][j] + cost;
                }
            }
            dp[i][j] = mn;
        }
    }
    return dp[0][N-1];
}

```

MO optimization

```

inline int64_t gilbertOrder(int x, int y, int pow, int rotate) {
    if (pow == 0) {
        return 0;
    }
    int hpow = 1 << (pow-1);
    int seg = (x < hpow) ? (
        (y < hpow) ? 0 : 3
    ) : (
        (y < hpow) ? 1 : 2
    );
};

```

```

    seg = (seg + rotate) & 3;
    const int rotateDelta[4] = {3, 0, 0, 1};
    int nx = x & (x ^ hpow), ny = y & (y ^ hpow);
    int nrot = (rotate + rotateDelta[seg]) & 3;
    int64_t subSquareSize = int64_t(1) << (2*pow - 2);
    int64_t ans = seg * subSquareSize;
    int64_t add = gilbertOrder(nx, ny, pow-1, nrot);
    ans += (seg == 1 || seg == 2) ? add : (subSquareSize - add - 1);
    return ans;
}
// sort by gilbertOrder(ql, qr, 20, 0)

```

Interactor

```

from subprocess import PIPE, Popen
import threading, sys, time

A = "a.exe"
B = "inter.exe"
TIMEOUT = 1.0 # seconds

def pipe(src, dst, tag):
    for line in iter(src.readline, b''):
        print(f"[{tag}] {line.decode(errors='ignore')}", end="")
        try:
            dst.write(line)
            dst.flush()
        except: return
    src.close()

pa = Popen(A, stdin=PIPE, stdout=PIPE, stderr=PIPE)
pb = Popen(B, stdin=PIPE, stdout=PIPE, stderr=PIPE)
threads = [
    threading.Thread(target=pipe, args=(pa.stdout, pb.stdin, "A→B")),
    threading.Thread(target=pipe, args=(pb.stdout, pa.stdin, "B→A")),
    threading.Thread(target=pipe, args=(pa.stderr, sys.stdout.buffer, "A err")),
    threading.Thread(target=pipe, args=(pb.stderr, sys.stdout.buffer, "B err")),
]
for t in threads:
    t.daemon = True
    t.start()
start = time.time()
while pa.poll() is None and pb.poll() is None:
    if time.time() - start > TIMEOUT:
        print("[TIMEOUT]")
        pa.kill()
        pb.kill()
        break
    time.sleep(0.05)
print("Exit codes:", pa.poll(), pb.poll())

```

Stress testing

```

@echo off
:loop
python comm.py > in && a < in
IF %ERRORLEVEL%==0 GOTO loop
echo Program exited with code %ERRORLEVEL%

```

			Divisors count											
< 10^k	number	divisors	2	3	5	7	11	13	17	19	23	29	31	37
1	6	4	1	1										
2	60	12	2	1	1									
3	840	32	3	1	1	1								
4	7560	64	3	3	1	1								
5	83160	128	3	3	1	1	1							
6	720720	240	4	2	1	1	1	1						
7	8648640	448	6	3	1	1	1	1	1					
8	73513440	768	5	3	1	1	1	1	1	1				
9	735134400	1344	6	3	2	1	1	1	1	1				
10	6983776800	2304	5	3	2	1	1	1	1	1	1			
11	97772875200	4032	6	3	2	2	1	1	1	1	1			
12	963761198400	6720	6	4	2	1	1	1	1	1	1	1		
13	9316358251200	10752	6	3	2	1	1	1	1	1	1	1	1	
14	97821761637600	17280	5	4	2	2	1	1	1	1	1	1	1	
15	866421317361600	26880	6	4	2	1	1	1	1	1	1	1	1	1
16	8086598962041600	41472	8	3	2	2	1	1	1	1	1	1	1	1
17	74801040398884800	64512	6	3	2	2	1	1	1	1	1	1	1	1
18	897612484786617600	103680	8	4	2	2	1	1	1	1	1	1	1	1

< 10^k	prime	# of prime	< 10^k	prime
1	7	4	10	9999999967
2	97	25	11	99999999977
3	997	168	12	999999999989
4	9973	1229	13	9999999999971
5	99991	9592	14	99999999999973
6	999983	78498	15	999999999999989
7	9999991	664579	16	9999999999999937
8	99999989	5761455	17	9999999999999997
9	999999937	50847534	18	99999999999999989